

Node.js & Express

From JavaScript Fundamentals to Backend Internals — Concepts, Internals, and Runnable Examples

This guide picks up where your JavaScript and React notes left off. React runs your JS *in the browser*, on top of the DOM. Node.js runs the same JS engine (V8) *outside* the browser, giving JavaScript access to the filesystem, network, and OS. Express is a thin, unopinionated layer on top of Node's raw HTTP module that makes building APIs practical. Every section below connects a JS concept you already know to how Node/Express actually use it under the hood, followed by runnable examples.

Contents

1. What Node.js Actually Is (and Isn't)
2. The Event Loop — Node's Core Superpower
3. Modules: CommonJS vs ES Modules
4. Core Modules: fs, path, os, events
5. Building a Server with the Raw http Module
6. npm, package.json, and Semantic Versioning
7. Introducing Express — Why It Exists
8. Routing in Express
9. Middleware — The Heart of Express
10. Request & Response Objects
11. Handling JSON, Forms, and File Uploads
12. Error Handling Patterns
13. Connecting to a Database (MongoDB + Mongoose)
14. Environment Variables & Configuration

- 15. Authentication Basics (JWT)
- 16. Project Structure for Real Apps
- 17. Building a Complete REST API (Worked Example)
- 18. Async Patterns in Node — Callbacks to Promises to async/await
- 19. Testing & Debugging Basics
- 20. Interview Question Bank

1. What Node.js Actually Is (and Isn't)

Node.js is **not a language** and not a framework — it's a JavaScript **runtime**: Google's V8 engine (the same one inside Chrome) plus a set of C++ bindings (libuv) that expose OS-level features — files, sockets, timers, processes — to JavaScript. In the browser, JS is sandboxed and can only touch the DOM, fetch, and a few Web APIs. Node removes the browser sandbox and swaps those Web APIs for server-side ones: `fs`, `http`, `process`, `net`, and so on.

Why this matters coming from React

- There is no `window` or `document` in Node — instead there's `global` and `process`.
- React code runs once per user, in their browser. Node/Express code runs once, continuously, on a server, serving *every* user — so a blocking mistake affects everyone, not just one tab.
- Both share the same async mental model: the event loop, promises, and `async/await` work identically — this is the single biggest transferable skill.

check_env.js

```
// check_env.js — proof you're in Node, not the browser
console.log(typeof window);    // undefined
console.log(typeof process);   // 'object'
console.log(process.version);   // e.g. v20.11.0
console.log(process.platform);  // 'linux', 'darwin', 'win32'
```

2. The Event Loop — Node's Core Superpower

Node is **single-threaded** for your JS code, but it's not blocking. It achieves concurrency with the **event loop**: expensive I/O (disk, network, DB queries) is handed off to the OS or a thread pool (libuv), and Node keeps executing other code while it waits. When the I/O finishes, a callback is queued and the event loop picks it up.

The phases, simplified

- **Timers** — runs callbacks scheduled by `setTimeout/setInterval`.
- **Pending callbacks** — I/O callbacks deferred from the previous cycle.
- **Poll** — retrieves new I/O events; executes I/O-related callbacks (this is where most work happens).
- **Check** — runs `setImmediate()` callbacks.
- **Close callbacks** — e.g. `socket.on('close', ...)`.

event_loop_order.js

```
console.log('1: start');

setTimeout(() => console.log('2: timeout (macrotask)'), 0);

Promise.resolve().then(() => console.log('3: promise (microtask)'));

fs.readFile(__filename, () => {
  console.log('4: fs callback (I/O)');
});

console.log('5: end');

// Output order:
// 1: start
// 5: end
// 3: promise (microtask)  <- microtasks always drain before the next macrotask
// 2: timeout (macrotask)
// 4: fs callback (I/O)
```

■ **Note:** Microtasks (Promises, `process.nextTick`) always run to completion before the event loop moves to the next phase — this is exactly why `await` 'pauses' a function without blocking the whole process.

3. Modules: CommonJS vs ES Modules

Node originally used **CommonJS** (`require/module.exports`), which is synchronous and loads modules on demand. Modern Node also supports **ES Modules** (`import/export`) — the same syntax you already use in React.

CommonJS

```
// ---- CommonJS (default in .js files) ----
// math.js
function add(a, b) { return a + b; }
module.exports = { add };

// app.js
const { add } = require('./math');
console.log(add(2, 3)); // 5
```

```
// ---- ES Modules ----
// Enable by adding "type": "module" to package.json, or use .mjs files

// math.mjs
export function add(a, b) { return a + b; }

// app.mjs
import { add } from './math.mjs';
console.log(add(2, 3)); // 5
```

	CommonJS	ES Modules
Syntax	require / module.exports	import / export
Loading	Synchronous	Asynchronous (can be top-level await)
File hint	default in .js	.mjs, or type:module in package.json
Tree-shaking	No	Yes (helps bundlers eliminate dead code)

4. Core Modules: fs, path, os, events

These ship with Node — no install needed. You'll use them constantly.

fs + path

```
const fs = require('fs');
const path = require('path');

// Async (non-blocking) — preferred
fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});

// Promise-based (cleanest, works with async/await)
const fsp = require('fs/promises');
async function readData() {
  const data = await fsp.readFile('data.txt', 'utf8');
  console.log(data);
}

// path — always use this instead of string concatenation
const filePath = path.join(__dirname, 'uploads', 'photo.png');
console.log(path.extname(filePath)); // '.png'
```

events (EventEmitter)

```
// events – the pattern Express itself is built on internally
const EventEmitter = require('events');

class OrderQueue extends EventEmitter {}
const queue = new OrderQueue();

queue.on('newOrder', (order) => {
  console.log('Processing order:', order.id);
});

queue.emit('newOrder', { id: 101 });
```

■ **Note:** `fs.readFileSync` blocks the entire event loop until the file is read — fine in a one-off script, dangerous in a server handling many requests. Prefer the async or promise-based versions in any Express route.

5. Building a Server with the Raw http Module

Before touching Express, build one server by hand — it demystifies everything Express does for you.

server.js — raw Node

```
const http = require('http');

const server = http.createServer((req, res) => {
  console.log(req.method, req.url);

  if (req.url === '/' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Home page');
  } else if (req.url === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('About page');
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('Not found');
  }
});

server.listen(3000, () => console.log('Listening on port 3000'));
```

Notice the manual `if/else` routing and manual header-setting. This is exactly the boilerplate Express eliminates — Express is essentially a router + middleware pipeline wrapped around this same `http.createServer` call.

6. npm, package.json, and Semantic Versioning

common npm commands

```
npm init -y           # create package.json
npm install express    # add a dependency (saved to "dependencies")
npm install -D nodemon # dev-only dependency (saved to "devDependencies")
npm install            # install everything from package.json
npm uninstall express
npm run <script-name>  # run a script defined in package.json
npx <package>          # run a package without installing it globally
```

package.json

```
{
  "name": "my-api",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "dependencies": {
    "express": "^4.19.2",
    "mongoose": "^8.4.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

Semantic versioning: MAJOR.MINOR.PATCH

- **^4.19.2** — accepts 4.x.x updates (safe minor/patch upgrades), never jumps to 5.0.0.
- **~4.19.2** — accepts only patch updates (4.19.x).
- **4.19.2** exact — locks the version completely.
- `package-lock.json` records exact resolved versions — always commit it.

7. Introducing Express — Why It Exists

Express is a minimal web framework built on top of Node's `http` module. It gives you clean routing, a middleware pipeline, and helpers for responses (`res.json()`, `res.status()`, etc.) without hiding Node underneath.

Minimal Express app

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

Compare this to the raw `http` version in Section 5 — same underlying server, far less boilerplate.

8. Routing in Express

basic routes

```
app.get('/users', (req, res) => { /* list users */ });
app.post('/users', (req, res) => { /* create user */ });
app.put('/users/:id', (req, res) => { /* replace user */ });
app.patch('/users/:id', (req, res) => { /* partial update */ });
app.delete('/users/:id', (req, res) => { /* delete user */ });

// Route params
app.get('/users/:id', (req, res) => {
  res.json({ userId: req.params.id });
});

// Query strings: GET /search?q=node&limit=10
app.get('/search', (req, res) => {
  const { q, limit } = req.query;
  res.json({ q, limit });
});
```

express.Router() — modular routes (like splitting React components)

modular routing

```
// routes/users.js
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => res.json({ msg: 'all users' }));
router.get('/:id', (req, res) => res.json({ msg: `user ${req.params.id}` }));

module.exports = router;

// index.js
const usersRouter = require('./routes/users');
app.use('/api/users', usersRouter); // GET /api/users, GET /api/users/:id
```

9. Middleware — The Heart of Express

A middleware function is any function with signature `(req, res, next)` that runs before the final route handler. Every request flows through a pipeline of these — think of it like a chain of `useEffects` that each get a chance to inspect or modify the request before it reaches your component.

custom middleware

```
// A simple logging middleware
function logger(req, res, next) {
  console.log(`${new Date().toISOString()} - ${req.method} ${req.url}`);
  next(); // MUST call next() or the request hangs forever
}

app.use(logger); // applies to every route

// Route-specific middleware
function requireAuth(req, res, next) {
  const token = req.headers.authorization;
  if (!token) return res.status(401).json({ error: 'No token provided' });
  next();
}

app.get('/dashboard', requireAuth, (req, res) => {
  res.json({ msg: 'secret dashboard data' });
});
```

Common built-in / third-party middleware

commonly-used middleware

```
app.use(express.json());           // parse JSON request bodies -> req.body
app.use(express.urlencoded({ extended: true })); // parse form data
app.use(express.static('public')); // serve static files (css, images)

const cors = require('cors');
app.use(cors());                  // allow cross-origin requests (needed for React frontend)
```

■ **Note:** If you're calling this API from a React app running on a different port (e.g. localhost:5173 hitting localhost:3000), you need cors middleware or the browser will block the request.

10. Request & Response Objects

req.<i>property</i>	What it holds
req.params	Route params, e.g. /users/:id -> req.params.id
req.query	Query string, e.g. ?limit=5 -> req.query.limit
req.body	Parsed request body (needs express.json() first)
req.headers	All HTTP headers, e.g. req.headers.authorization
req.method	'GET', 'POST', etc.

response helpers

```
res.send('plain text or html');
res.json({ key: 'value' });           // sets Content-Type automatically
res.status(404).json({ error: 'Not found' });
res.redirect('/login');
res.sendFile(path.join(__dirname, 'index.html'));
res.set('X-Custom-Header', 'value');
```

11. Handling JSON, Forms, and File Uploads

JSON body handling

```
app.use(express.json());

app.post('/api/users', (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ error: 'name and email are required' });
  }
  // ...save user
  res.status(201).json({ id: 1, name, email });
});
```

file upload with multer

```
// File uploads – multer is the standard middleware
const multer = require('multer');
const upload = multer({ dest: 'uploads/' });

app.post('/api/avatar', upload.single('avatar'), (req, res) => {
  res.json({ filename: req.file.filename, size: req.file.size });
});
```

12. Error Handling Patterns

Express error-handling middleware has a special 4-argument signature: `(err, req, res, next)`. It must be defined **after** all other routes and middleware.

```
// Wrap async route handlers so rejected promises don't crash the process
function asyncHandler(fn) {
  return (req, res, next) => Promise.resolve(fn(req, res, next)).catch(next);
}

app.get('/api/users/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) {
    const err = new Error('User not found');
    err.status = 404;
    throw err;
  }
  res.json(user);
})));

// Centralized error handler — must have 4 args, placed LAST
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({ error: err.message || 'Server error' });
});
```

■ **Note:** In plain callback-style Node code, an uncaught error inside an async operation can crash the entire server. In Express route handlers using `async/await`, an unhandled rejected promise silently hangs the request unless you catch it — hence the `asyncHandler` wrapper pattern above.

13. Connecting to a Database (MongoDB + Mongoose)

models/User.js

```
const mongoose = require('mongoose');

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch((err) => console.error('Connection error:', err));

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  createdAt: { type: Date, default: Date.now },
});

const User = mongoose.model('User', userSchema);

module.exports = User;
```

routes/users.js — CRUD with Mongoose

```
const User = require('../models/User');

router.post('/', asyncHandler(async (req, res) => {
  const user = await User.create(req.body);
  res.status(201).json(user);
}));

router.get('/', asyncHandler(async (req, res) => {
  const users = await User.find();
  res.json(users);
}));

router.get('/:id', asyncHandler(async (req, res) => {
  const user = await User.findById(req.params.id);
  if (!user) return res.status(404).json({ error: 'Not found' });
  res.json(user);
}));
```

If you used Supabase for the running-club project, the same pattern applies with the Supabase client instead of Mongoose — swap the model layer, keep the route/middleware structure identical.

14. Environment Variables & Configuration

dotenv usage

```
// .env (never commit this file – add it to .gitignore)
PORT=3000
MONGO_URI=mongodb://localhost:27017/mydb
JWT_SECRET=some-long-random-string

// index.js
require('dotenv').config();
const PORT = process.env.PORT || 3000;
```

■ **Note:** Just like a React app shouldn't hardcode API keys into components, a Node server should never hardcode secrets (DB passwords, JWT secrets) into source files — always read them from `process.env`.

15. Authentication Basics (JWT)

JWT auth flow

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

// Signup – hash the password before saving
router.post('/signup', asyncHandler(async (req, res) => {
  const hashed = await bcrypt.hash(req.body.password, 10);
  const user = await User.create({ ...req.body, password: hashed });
  res.status(201).json({ id: user._id });
}));

// Login – verify password, issue a token
router.post('/login', asyncHandler(async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  const valid = user && await bcrypt.compare(req.body.password, user.password);
  if (!valid) return res.status(401).json({ error: 'Invalid credentials' });

  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '1d' });
  res.json({ token });
}));

// Middleware to protect routes
function requireAuth(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1]; // "Bearer <token>"
  if (!token) return res.status(401).json({ error: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
}
```


16. Project Structure for Real Apps

typical layout

```
my-api/  
|-- index.js           # entry point - creates app, starts server  
|-- .env  
|-- package.json  
|-- config/  
|   |-- db.js          # DB connection logic  
|-- models/  
|   |-- User.js  
|-- routes/  
|   |-- users.js  
|   |-- auth.js  
|-- middleware/  
|   |-- requireAuth.js  
|   |-- errorHandler.js  
|-- controllers/  
|   |-- userController.js # route logic pulled out of route files
```

This mirrors why you split React apps into components/, hooks/, routes/ — same principle, same payoff: each file has one job, and routes stay thin by delegating to controllers.

17. Building a Complete REST API (Worked Example)

A minimal but complete task-manager API, assembled from everything above.

index.js

```
// index.js  
require('dotenv').config();  
const express = require('express');  
const cors = require('cors');  
const mongoose = require('mongoose');  
const tasksRouter = require('./routes/tasks');  
const errorHandler = require('./middleware/errorHandler');  
  
const app = express();  
  
app.use(cors());  
app.use(express.json());  
app.use('/api/tasks', tasksRouter);  
app.use(errorHandler); // always last  
  
mongoose.connect(process.env.MONGO_URI).then(() => {  
  app.listen(process.env.PORT || 3000, () =>  
    console.log(`Server running on port ${process.env.PORT || 3000}`)  
  });  
});
```

models/Task.js

```
// models/Task.js
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema({
  title: { type: String, required: true },
  done: { type: Boolean, default: false },
}, { timestamps: true });

module.exports = mongoose.model('Task', taskSchema);
```

routes/tasks.js

```
// routes/tasks.js
const express = require('express');
const router = express.Router();
const Task = require('../models/Task');
const asyncHandler = require('../middleware/asyncHandler');

router.get('/', asyncHandler(async (req, res) => {
  res.json(await Task.find());
}));

router.post('/', asyncHandler(async (req, res) => {
  const task = await Task.create({ title: req.body.title });
  res.status(201).json(task);
}));

router.patch('/:id', asyncHandler(async (req, res) => {
  const task = await Task.findByIdAndUpdate(req.params.id, req.body, { new: true });
  if (!task) return res.status(404).json({ error: 'Task not found' });
  res.json(task);
}));

router.delete('/:id', asyncHandler(async (req, res) => {
  await Task.findByIdAndDelete(req.params.id);
  res.status(204).send();
}));

module.exports = router;
```

This is the exact shape a React frontend would call with `fetch('/api/tasks')` — the same REST conventions you'd wire up to use `Effect` + `useState` on the client.

18. Async Patterns in Node — Callbacks to Promises to async/await

three eras of async Node code

```
// 1. Callback style (older Node APIs) — "callback hell" risk
fs.readFile('a.txt', 'utf8', (err, a) => {
  fs.readFile('b.txt', 'utf8', (err, b) => {
    console.log(a + b);
  });
});

// 2. Promises — flatten with .then chains
fsp.readFile('a.txt', 'utf8')
  .then((a) => fsp.readFile('b.txt', 'utf8').then((b) => a + b))
  .then(console.log)
  .catch(console.error);

// 3. async/await — same Promises, readable as sync code (identical to React)
async function combine() {
  try {
    const a = await fsp.readFile('a.txt', 'utf8');
    const b = await fsp.readFile('b.txt', 'utf8');
    console.log(a + b);
  } catch (err) {
    console.error(err);
  }
}
```

This is exactly the same ladder you climbed for React data-fetching — `.then()` chains for API calls versus `await fetch()` inside an async function. Node just applies it to files, databases, and network sockets instead of HTTP responses.

19. Testing & Debugging Basics

supertest + jest

```
npm install -D jest supertest

// tasks.test.js
const request = require('supertest');
const app = require('../app'); // export app separately from app.listen()

test('GET /api/tasks returns 200 and an array', async () => {
  const res = await request(app).get('/api/tasks');
  expect(res.statusCode).toBe(200);
  expect(Array.isArray(res.body)).toBe(true);
});
```

- `console.log` is fine for quick checks; for real debugging, run `node --inspect index.js` and attach Chrome DevTools or VS Code's debugger.
- nodemon auto-restarts the server on file changes during development — the backend equivalent of React's hot reload.
- Log request/response timing with a small middleware to catch slow routes early.

20. Interview Question Bank

Q: Is Node.js single-threaded?

Your JS callback code runs on a single thread, but Node offloads I/O and some CPU work to libuv's thread pool and the OS, so the process as a whole handles concurrency without blocking.

Q: What's the difference between `process.nextTick()` and `setImmediate()`?

`process.nextTick()` queues a callback to run immediately after the current operation, before the event loop continues — even before Promises. `setImmediate()` runs in the 'check' phase, after I/O callbacks.

Q: What does middleware's `next()` actually do?

It hands control to the next middleware/handler in the stack. If you don't call it (or send a response), the request hangs forever with no response sent.

Q: Why does the error-handling middleware need exactly 4 arguments?

Express inspects a middleware function's arity (parameter count) to decide whether it's a normal middleware (3 args) or an error handler (4 args) — it's how Express routes errors to the right place.

Q: What's the difference between `req.params`, `req.query`, and `req.body`?

`params` come from the URL path (`/users/:id`), `query` comes from the URL's query string (`?limit=5`), and `body` comes from the request payload — only populated if a body-parsing middleware like `express.json()` ran first.

Q: How would you prevent one crashed route from taking down the whole server?

Wrap async handlers so rejected promises are passed to `next(err)` instead of throwing unhandled, keep a centralized error-handling middleware last in the stack, and consider a process manager (PM2) that restarts the process on an uncaught exception.

Q: Why use environment variables instead of hardcoding config?

They keep secrets out of source control, let the same code run with different settings across dev/staging/production, and match the twelve-factor app principle of separating config from code.

Q: What is CORS and why does a React app calling an Express API need it configured?

CORS (Cross-Origin Resource Sharing) is a browser security policy that blocks requests to a different origin (domain/port) unless the server explicitly allows it via response headers — the `cors` middleware adds those headers automatically.

End of notes. Suggested next step: rebuild the running-club backend's Supabase routes using this Router + `asyncHandler` + centralized error-handler pattern for consistency across your projects.